

PROJECT REPORT Project Title: ComicCraft - AI Comic Story Creator using Gemini Models

1. Team Information

- **Team ID:** SWTID-2026-7486
- **Team Size:** 5
- **Team Leader:** Diwan Oli M
- **Team Members:** Siva, Akashdeepan, Abdul Bazith K, Mohamed Ibrahim

2. Abstract

ComicCraft is an intelligent, AI-powered web platform engineered to instantly generate custom comic books from basic text prompts. Understanding that traditional comic creation demands significant artistic expertise and time, this solution utilizes Generative AI to automate the entire pipeline. By analyzing user inputs like character names, storyline settings, and art styles, the application builds a comprehensive narrative and pairs it with matching AI-generated illustrations. Powered by a high-speed FastAPI backend, ComicCraft guarantees a smooth transition from text input to a fully formatted, downloadable PDF output.

3. Project Objectives

- To engineer a user-friendly web interface that efficiently captures narrative ideas and stylistic choices.
- To utilize Google's Gemini models (Flash and Pro) for structuring sequential panels and generating dynamic dialogues.
- To deploy Hugging Face Diffusers (Stable Diffusion) to dynamically create high-resolution, comic-style imagery.
- To automate the document layout process using the FPDF library, enabling users to download a sequential, ready-to-read comic book instantly.

4. Technology Stack

- **Programming Language:** Python 3.10

- **Backend Framework:** FastAPI, Uvicorn Server
- **AI Text Generation:** Google Gemini API (gemini-1.5-flash, gemini-1.5-pro)
- **AI Image Generation:** Hugging Face Inference API (runwayml/stable-diffusion-v1-5)
- **PDF Export Logic:** FPDF Library
- **Frontend UI:** Jinja2 Templates, HTML5, CSS3

5. System Architecture & Data Flow

The application is structured on a fast, decoupled client-server model integrating multiple external AI services:

1. **User Request:** The individual submits their story prompt and style preferences via the frontend HTML form.
2. **Backend Processing:** FastAPI intercepts the POST request and triggers the sequential AI generation tasks.
3. **LLM Orchestration:** Google Gemini processes the prompt to output a structured JSON outline, followed by complete story dialogues.
4. **Image Rendering:** The Stable Diffusion model interprets the scene descriptions to generate corresponding panel artwork.
5. **Layout Assembly:** The system's layout builder aligns the generated narrative text with the downloaded images.
6. **PDF Generation & UI Render:** FPDF compiles the offline document, while Jinja2 updates the browser with a digital preview and a download link.

Data Flow: Web Form → FastAPI (routes) → Gemini & Hugging Face APIs → FPDF Compiler → Rendered Preview UI.

6. Key Code Snippets

A. Dual-Engine Image Generation (image_handler.py)

```
import urllib.request, json, os, io
```

```
from PIL import Image
```

```

def generate_panel_image(prompt, filepath):
    hf_token = os.getenv("HF_API_KEY")

    url = "https://api-inference.huggingface.co/models/runwayml/stable-diffusion-
v1-5"

    headers = {"Authorization": f"Bearer {hf_token}", "Content-Type":
"application/json"}

    payload = {"inputs": f"{prompt}, highly detailed comic book illustration"}

    try:
        req = urllib.request.Request(url, data=json.dumps(payload).encode('utf-8'),
headers=headers)

        response = urllib.request.urlopen(req, timeout=45)

        img = Image.open(io.BytesIO(response.read())).convert('RGB')

        img.save(filepath, format='JPEG', quality=95)

        return filepath

    except Exception as e:

        return generate_fallback_image(filepath) # Triggers PIL fallback on failure

```

B. Application Routing (main_routes.py)

```

@router.post("/create-comic", response_class=HTMLResponse)

async def create_comic(request: Request, story_prompt: str = Form(...), character:
str = Form(...)):

    # AI Pipeline Execution

```

```

full_prompt = f"{story_prompt}. Lead Character: {character}"

panel_outline = get_gemini_outline(full_prompt)

narrative = get_gemini_story(panel_outline)


# Generate Images iteratively

image_paths = []

for i, panel in enumerate(panel_outline):

    path = generate_panel_image(panel["image_prompt"], f"panel_{i}.jpg")

    image_paths.append(path)


# Compile Layout and Export PDF

final_layout = construct_layout(image_paths, narrative, panel_outline)

exported_pdf = compile_pdf(final_layout)


return templates.TemplateResponse("preview.html", {"request": request,
"layout": final_layout, "pdf": exported_pdf})

```

7. Output Screenshots

- **Screenshot 1: Home Page / User Input Form** (*Inge unga 1st screenshot-ah paste pannunga*)
- **Screenshot 2: Generated Comic Preview (Top Section)** (*Inge unga 2nd screenshot-ah paste pannunga*)
- **Screenshot 3: Comic Preview (Bottom Section)** (*Inge unga 3rd screenshot-ah paste pannunga*)

- **Screenshot 4: Downloaded PDF Document** (*Inge unga 4th screenshot-ah paste pannunga*)

8. Conclusion ComicCraft effectively highlights the power of combining modern Python web frameworks with advanced Generative AI capabilities. By abstracting the complexities of prompt engineering and manual image formatting, the application provides a highly accessible toolkit for digital storytelling. The successful implementation of automated routing, dual-model text logic, and dynamic PDF generation proves the system's viability as a robust, scalable creative technology solution.